

MDTS4313_737_3

SULAGNA ROY

2026-08-16

Problem Set 2 : KNN Smoother

1. Write the following lines of code in R.

```
set . seed (1234)
```

```
X=runif(100 ,5 ,15)
```

```
Y=5sin(X)+23cos(X)^2+rnorm(100 ,0 ,5)
```

```
Sim.1=data.frame(X=X,Y=Y)
```

```
reg=function(x){5sin(x)+23cos(x)^2}
```

The function “reg” represents the truth (i.e., the regression function) of the simulated dataset.

Seed for Reproducible Outputs

```
set.seed (1234)
```

100 obs.s on predictor X from U(5,15)

```
X=runif(100,5,15)
```

100 obs. on response Y (errors~N(mean=0,sd=5))

```
Y=(5*sin(X))+(23*cos(X)^2)+rnorm(100,0,5)
```

Full Data Set

```
Sim.1=data.frame(X=X,Y=Y)
```

100 obs. on the true regression function.

```
reg=function(x){5*sin(x)+23*cos(x)^2}
```

2. Divide the X values into parts. Call the first 80 values the training test. Denote them by X.train. The rest of the 20 values constitute the test set. Call them X.test.

Training Set of Predictor X

```
X.train=X[1:80]
```

Test Set of Predictor X

```
X.test=X[81:100]
```

3. Divide the Y values in training set (Y.train) and test set (Y.test) in the similar way.

Training Set of Response Y

```
Y.train=Y[1:80]
```

Test Set of Response Y

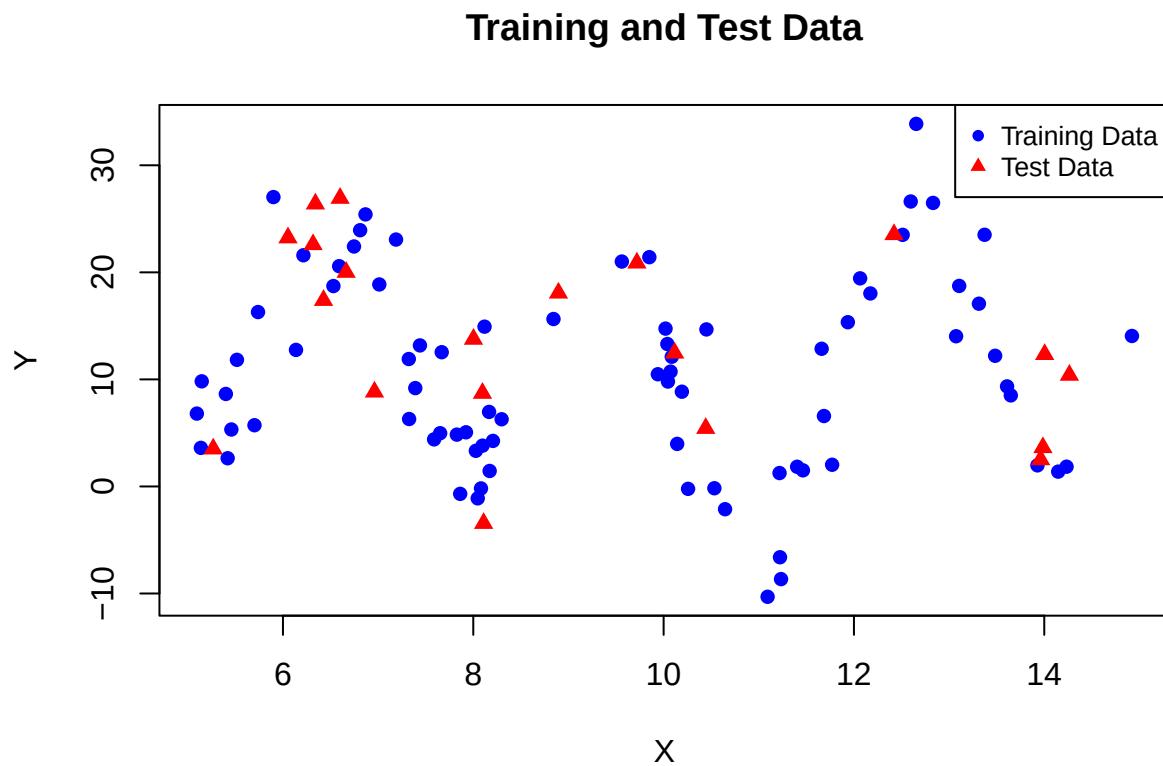
```
Y.test=Y[81:100]
```

4. Plot the data such that the points corresponding to the training set and the test set are represented with different symbols.

```
# Filled circle for training
plot(X.train,Y.train,pch = 16,col="blue",
     xlim = range(X),ylim=range(Y),
     xlab="X",ylab="Y",main="Training and Test Data")

# Add test observations(# Triangle for test )
points(X.test,Y.test,pch=17,col="red")

# Add legend
legend("topright",legend=c("Training Data","Test Data"),pch = c(16, 17),col = c("blue", "red"),cex=0.8)
```



5. Plot the regression function in the same graph.

```
# Create a sequence of X values for a smooth regression curve
x.grid=seq(min(X),max(X),length.out=500)

# Calculate the true regression function
y.true=reg(x.grid)

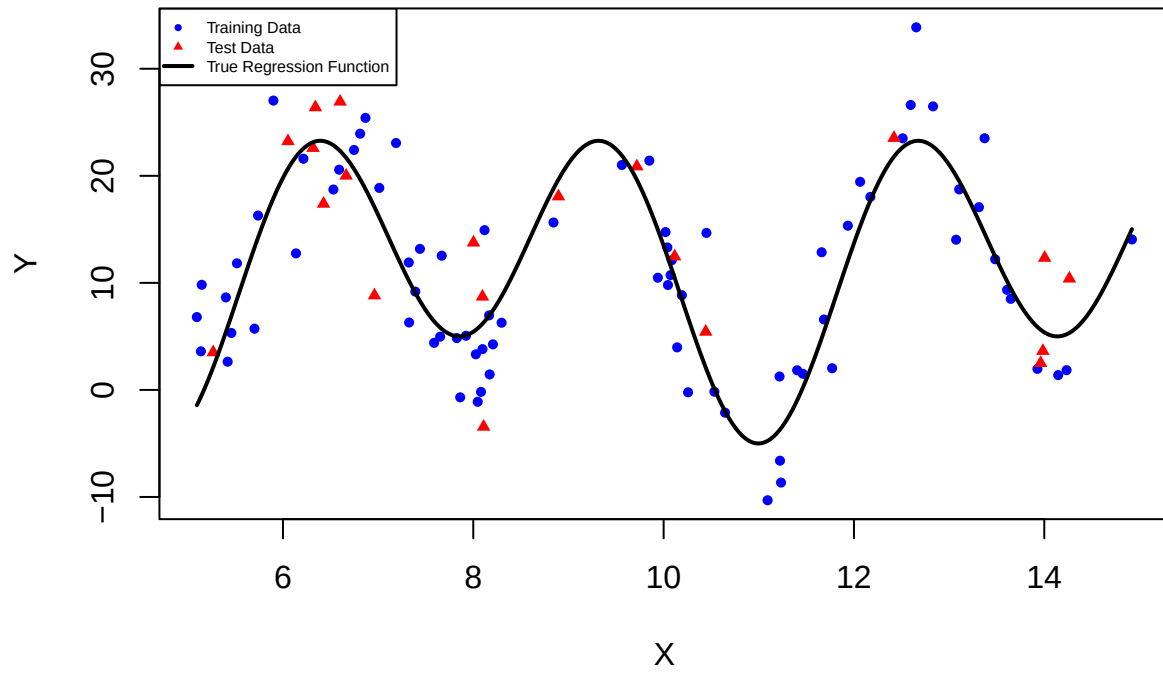
# Filled circle for training
plot(X.train,Y.train,pch = 16,col="blue",cex=0.7,
      xlim = range(X),ylim=range(Y),
      xlab="X",ylab="Y",main="Training and Test Data")

# Add test observations(# Triangle for test )
points(X.test,Y.test,pch=17,col="red",cex=0.7)

# Add true regression function
lines(x.grid,y.true,col="black",lwd = 2)

# Add legend
legend("topleft",
      legend=c("Training Data","Test Data","True Regression Function"),
      pch=c(16,17,NA),
      lty=c(NA,NA,1),
      col=c("blue","red","black"),
      lwd=c(NA,NA,2),
      cex=0.5)
```

Training and Test Data



6. Apply the KNN smoother algorithm on the training dataset with the following number of nearest neighbours (k)

- (a) $k=1$.
- (b) $k=2$.
- (c) $k=5$.
- (d) $k=10$.
- (e) $k=20$.
- (f) $k=40$.
- (g) $k=80$.

For each case, plot the estimates.

Comment on your findings (i.e., how do the estimates change with changing k ?)

```
# Values of k
k.values=c(1,2,5,10,20,40,80)

# Function for KNN smoother
knn.smooth=function(x.train,y.train,x.new,k)
{
  sapply(x.new,function(x0)
  {
    d=abs(x.train-x0)
    y.train[order(d)[1:k]] |> mean()
  })
}

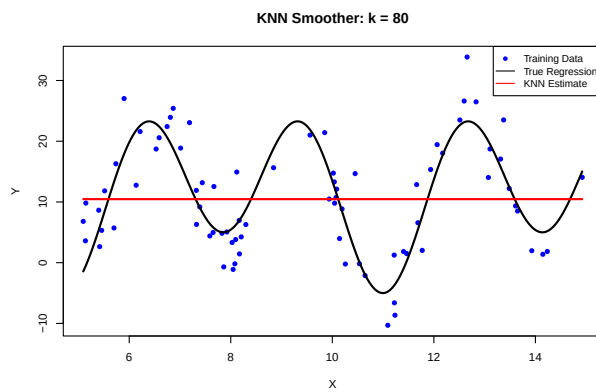
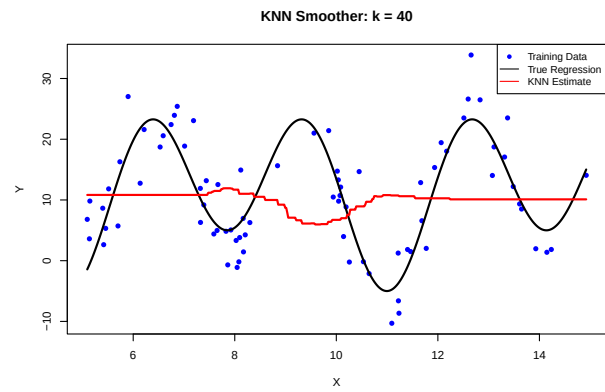
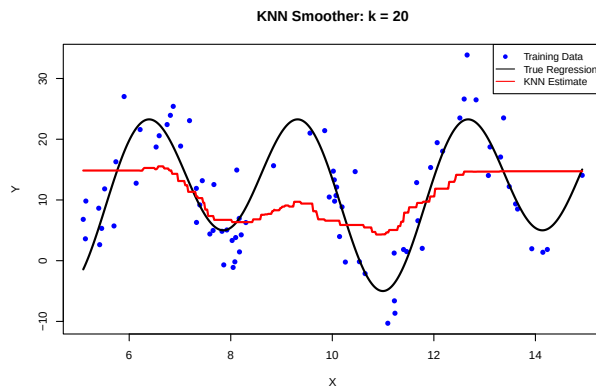
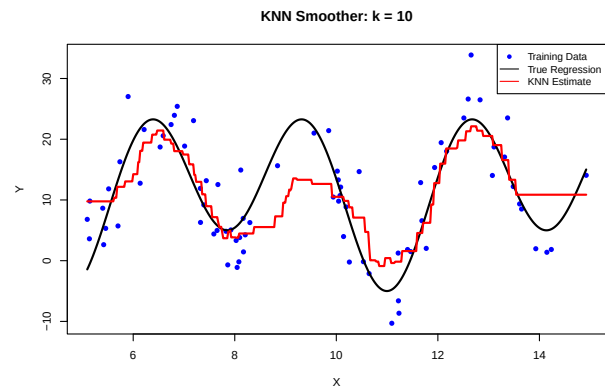
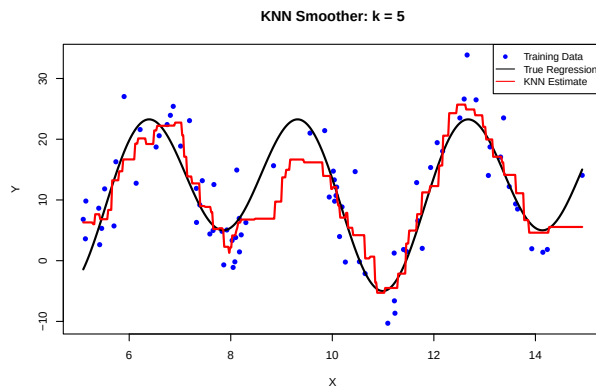
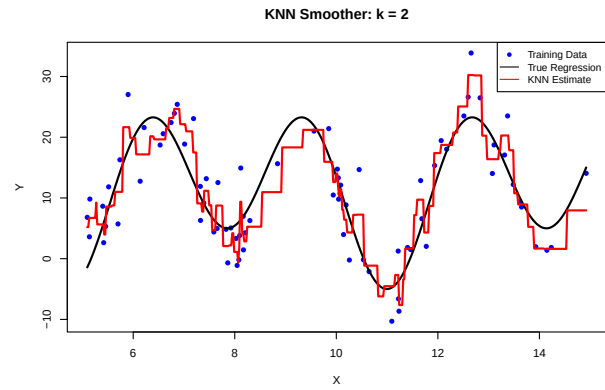
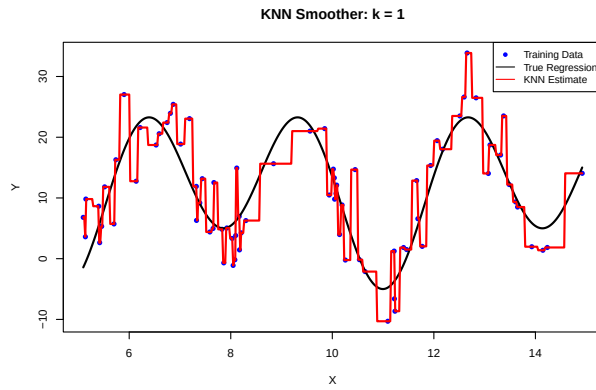
# Plot all KNN estimates
par(mfrow=c(4,2))

for(k in k.values)
{
  y.knn=knn.smooth(X.train,Y.train,x.grid,k)

  plot(X.train,Y.train,pch=16,col="blue",
        xlab="X",ylab="Y",
        main=paste("KNN Smoother: k =",k))

  lines(x.grid,reg(x.grid),lwd=2,col="black")
  lines(x.grid,y.knn,lwd=2,col="red")
}
```

```
legend("topright",  
      legend=c("Training Data","True Regression","KNN Estimate"),  
      pch=c(16,NA,NA),  
      lty=c(NA,1,1),  
      col=c("blue","black","red"),  
      cex=0.9)  
}  
  
par(mfrow=c(1,1))
```



Comment:

When k is small, the KNN estimate is more flexible and follows the individual observations closely. For $k=1$, the estimate is highly variable. As k increases, the estimate becomes smoother because more neighbouring observations are used. For very large k , the estimate becomes overly smooth and may fail to capture important variations in the true regression function. Thus, increasing k decreases variance but increases bias.

7. For each case, compute the training and test errors.

```
# Values of k
k.values=c(1,2,5,10,20,40,80)

# Store training and test errors
train.error=numeric(length(k.values))
test.error=numeric(length(k.values))

# Calculate errors for each k
for(i in seq_along(k.values))
{
  k=k.values[i]

  # Predictions for training data
  pred.train=knn.smooth(X.train,Y.train,X.train,k)

  # Predictions for test data
  pred.test=knn.smooth(X.train,Y.train,X.test,k)

  # Mean Squared Errors
  train.error[i]=mean((Y.train-pred.train)^2)
  test.error[i]=mean((Y.test-pred.test)^2)
}

# Display results
error.table=data.frame(
  k=k.values,
  Training.Error=train.error,
  Test.Error=test.error
)

error.table
```

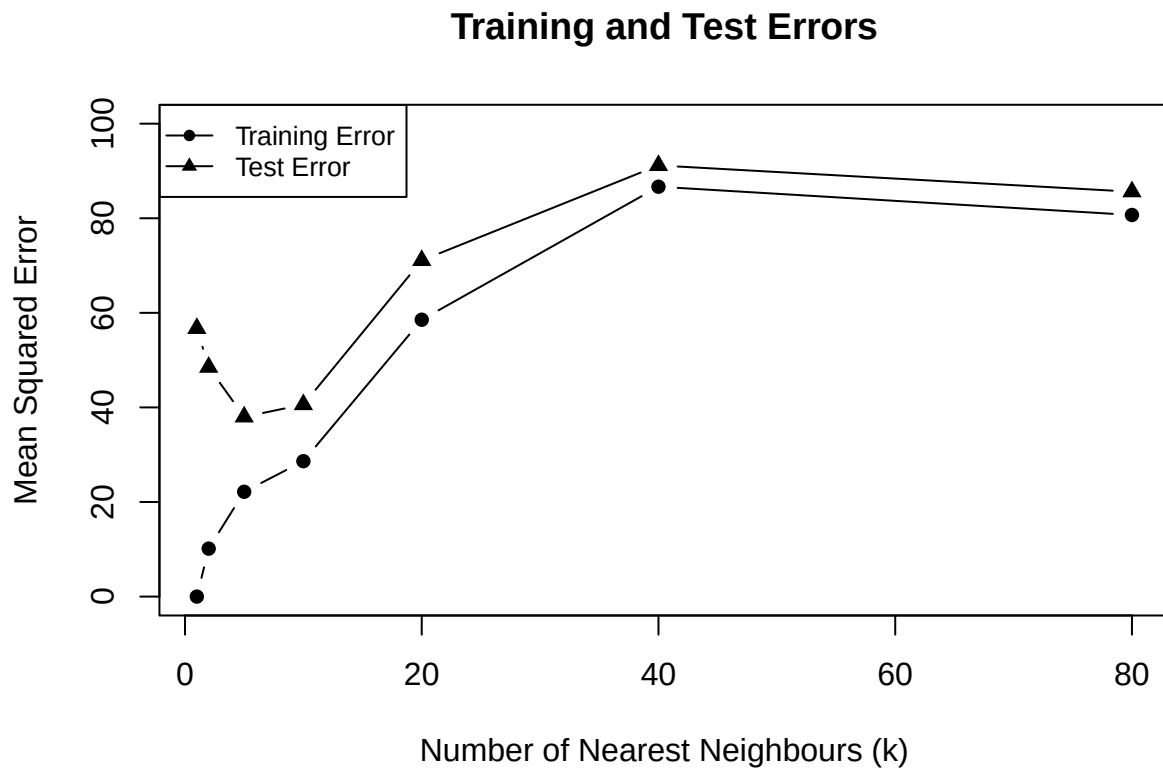
##	k	Training.Error	Test.Error
## 1	1	0.00000	56.71625
## 2	2	10.13799	48.50686
## 3	5	22.15921	38.00319
## 4	10	28.63438	40.60309
## 5	20	58.54042	71.08497
## 6	40	86.67047	91.17228

```
## 7 80      80.69600  85.61395
```

```
plot(k.values,train.error,type="b",pch=16,ylim=c(0,100),
     xlab="Number of Nearest Neighbours (k)",
     ylab="Mean Squared Error",
     main="Training and Test Errors")

lines(k.values,test.error,type="b",pch=17)

legend("topleft",
      legend=c("Training Error","Test Error"),
      pch=c(16,17),lty=1,
      cex=0.8)
```



Comment :

The training error is generally lowest for small values of k , particularly $k=1$, because the smoother is highly flexible. As k increases, the training error generally increases. The test error may initially decrease and then increase as k becomes too large. Therefore, an intermediate value of k may provide a better balance between bias and variance.

8. Use the `knnreg` function in the `caret` library in R to compare the results with your code.

```
library(caret)

## Warning: package 'caret' was built under R version 4.5.3

## Loading required package: ggplot2

## Warning: package 'ggplot2' was built under R version 4.5.2

## Loading required package: lattice

k.values=c(1,2,5,10,20,40,80)

caret.train.error=numeric(length(k.values))
caret.test.error=numeric(length(k.values))

# Create training data frame
train.data=data.frame(
  Y=Y.train,
  X=X.train
)

# Create test data frame
test.data=data.frame(
  X=X.test
)

for(i in seq_along(k.values))
{
  k=k.values[i]

  # Fit KNN model
  model=knnreg(Y~X,data=train.data,k=k)

  # Predictions
  pred.train=predict(model,train.data)
  pred.test=predict(model,test.data)

  # Errors
  caret.train.error[i]=mean((Y.train-pred.train)^2)
  caret.test.error[i]=mean((Y.test-pred.test)^2)
}

# Display results
caret.table=data.frame(
  k=k.values,
  Training.Error=caret.train.error,
```

```

    Test.Error=caret.test.error
  )

  caret.table

```

```

##      k Training.Error Test.Error
## 1  1      0.00000    56.71625
## 2  2     10.13799    48.50686
## 3  5     22.15921    38.00319
## 4 10     28.63438    40.60309
## 5 20     58.54042    71.08497
## 6 40     86.67047    91.17228
## 7 80     80.69600    85.61395

```

```

comparison=data.frame(
  k=k.values,
  Manual.Train=train.error,
  Caret.Train=caret.train.error,
  Manual.Test=test.error,
  Caret.Test=caret.test.error
)

comparison

```

```

##      k Manual.Train Caret.Train Manual.Test Caret.Test
## 1  1      0.00000      0.00000    56.71625    56.71625
## 2  2     10.13799     10.13799    48.50686    48.50686
## 3  5     22.15921     22.15921    38.00319    38.00319
## 4 10     28.63438     28.63438    40.60309    40.60309
## 5 20     58.54042     58.54042    71.08497    71.08497
## 6 40     86.67047     86.67047    91.17228    91.17228
## 7 80     80.69600     80.69600    85.61395    85.61395

```

Comment :

The results obtained using `knnreg()` should be very close to the results obtained from the manually implemented KNN smoother. Small differences, if any, may arise from implementation details. This confirms that the manual KNN smoothing procedure has been correctly implemented.

9. Repeat the procedure 50 times. Each time, the training set and the test set should be chosen randomly from the given sample.

Find the average of the training errors and the testing errors thus obtained for each case

and comment.

```
# Values of k
k.values=c(1,2,5,10,20,40,80)

# Number of repetitions
B=50

# Store errors
train.errors=matrix(0,B,length(k.values))
test.errors=matrix(0,B,length(k.values))

set.seed(1234)

for(b in 1:B)
{
  # Randomly select 80 observations for training
  train.id=sample(1:100,80)
  test.id=setdiff(1:100,train.id)

  x.train=X[train.id]
  y.train=Y[train.id]
  x.test=X[test.id]
  y.test=Y[test.id]

  for(i in seq_along(k.values))
  {
    k=k.values[i]

    # Training predictions
    pred.train=knn.smooth(x.train,y.train,x.train,k)

    # Test predictions
    pred.test=knn.smooth(x.train,y.train,x.test,k)

    # Errors
    train.errors[b,i]=mean((y.train-pred.train)^2)
    test.errors[b,i]=mean((y.test-pred.test)^2)
  }
}

# Average errors over 50 repetitions
average.errors=data.frame(
```

```

k=k.values,
Average.Training.Error=colMeans(train.errors),
Average.Test.Error=colMeans(test.errors)
)

```

```
average.errors
```

```

##      k Average.Training.Error Average.Test.Error
## 1  1          0.00000         49.32188
## 2  2         12.08941         39.45500
## 3  5         21.74001         32.22203
## 4 10         29.66768         37.53825
## 5 20         61.56919         67.32186
## 6 40         86.83521         88.32208
## 7 80         81.39883         81.72774

```

```

plot(k.values,
     average.errors$Average.Training.Error,
     type="b",pch=16,
     xlab="Number of Nearest Neighbours (k)",
     ylab="Average Mean Squared Error",
     main="Average Errors over 50 Repetitions")

```

```

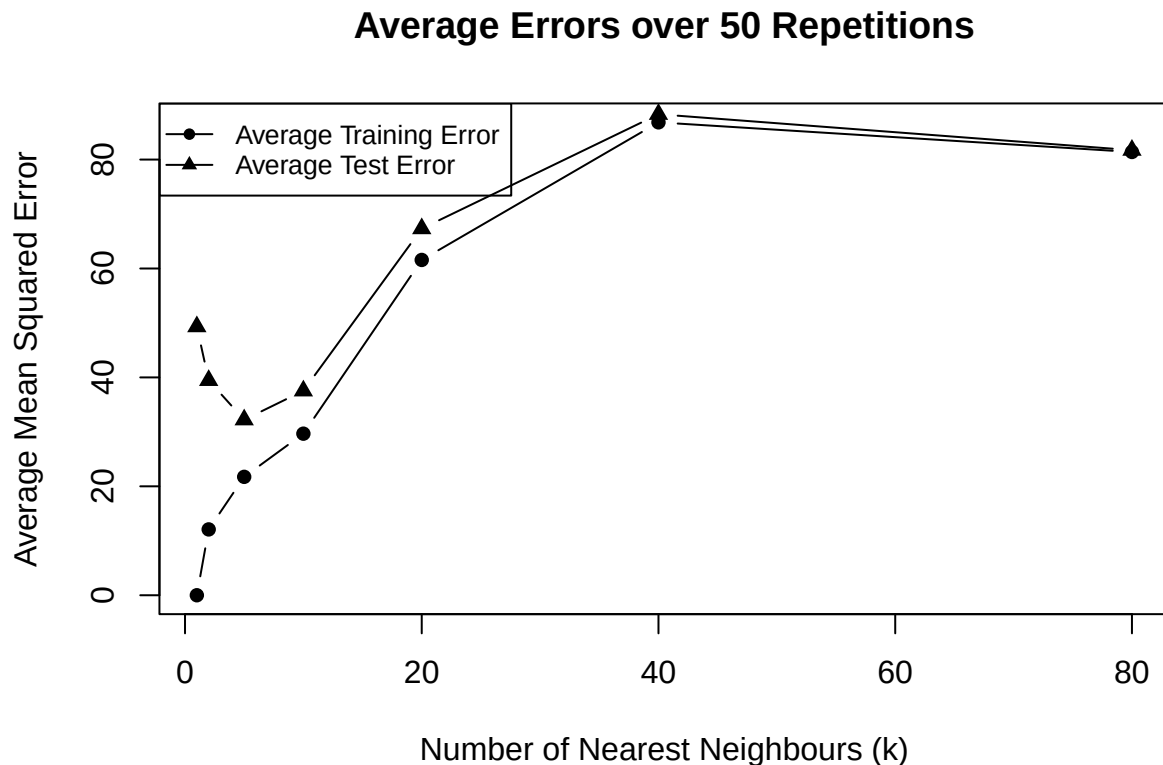
lines(k.values,
     average.errors$Average.Test.Error,
     type="b",pch=17)

```

```

legend("topleft",
     legend=c("Average Training Error",
              "Average Test Error"),
     pch=c(16,17),lty=1,cex=0.8)

```



Comment :

The average training error tends to increase as k increases because a larger neighbourhood produces a smoother and less flexible estimate. The average test error generally shows a bias-variance trade-off: very small values of k can have high variance, while very large values of k can have high bias. Hence, the value of k giving the smallest average test error can be considered a suitable choice.

10. Consider the “Boston” dataset in the “MASS” library in R and let the variable “medv” be the response (Y) and the variable “lstat” be the predictor (X).

Use 22-fold cross-validation method to find the best choice of k for applying the KNN smoother.

```
library(MASS)
library(caret)

# Load Boston data
```

```

data(Boston)

# Set seed for reproducibility
set.seed(1234)

# Values of k to compare
k.values=c(1,2,5,10,20,40,80)

# 22-fold cross-validation
ctrl=trainControl(
  method="cv",
  number=22
)

# Fit KNN model
knn.cv=train(
  medv~lstat,
  data=Boston,
  method="knn",
  tuneGrid=data.frame(k=k.values),
  trControl=ctrl,
  metric="RMSE"
)

# Cross-validation results
knn.cv$results

```

```

##      k      RMSE Rsquared      MAE  RMSESD RsquaredSD      MAESD
## 1  1  7.083981  0.5062433  5.244889  1.427981  0.16431305  1.0950898
## 2  2  6.352059  0.5677268  4.720380  1.079579  0.09913560  0.8481718
## 3  5  5.494815  0.6593598  4.064667  1.232902  0.10366228  0.8061542
## 4 10  5.366554  0.6733530  3.976625  1.181520  0.09835963  0.8072260
## 5 20  5.178291  0.6946954  3.835363  1.159800  0.09618119  0.7733403
## 6 40  5.106708  0.7037311  3.792335  1.152755  0.09068235  0.7415540
## 7 80  5.331728  0.6870411  3.897336  1.177987  0.07921057  0.7255062

```

```

# best k
knn.cv$bestTune

```

```

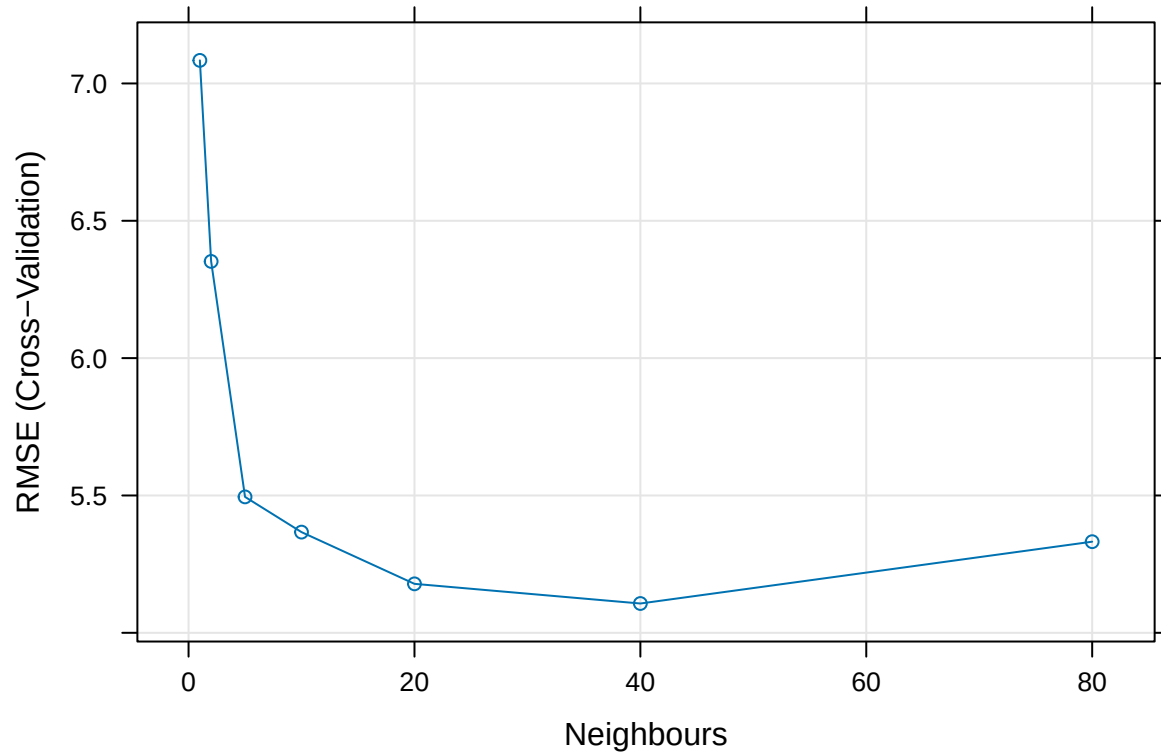
##      k
## 6 40

```

```

#Plot CV results
plot(knn.cv,xlab="Neighbours")

```



```
# Results
cv.results=knn.cv$results[,c("k", "RMSE", "Rsquared", "MAE")]

cv.results
```

##	k	RMSE	Rsquared	MAE
## 1	1	7.083981	0.5062433	5.244889
## 2	2	6.352059	0.5677268	4.720380
## 3	5	5.494815	0.6593598	4.064667
## 4	10	5.366554	0.6733530	3.976625
## 5	20	5.178291	0.6946954	3.835363
## 6	40	5.106708	0.7037311	3.792335
## 7	80	5.331728	0.6870411	3.897336

Comment :

The 22-fold cross-validation procedure evaluates the predictive performance of the KNN smoother for each candidate value of k . The best value of k is the one having the minimum cross-validated RMSE. This value provides the best predictive performance among the values of k considered.